



Summer Project Summary

The Magnetic Pendulum: Experimental and Theoretical Analysis

Sara Nargis

Danish Abbas Khan

Komal Parveen Sultana

Muhammad Husnain Ashfaq

Department: DPAM

Submitted to:

Dr. Zaheer Asghar

Pakistan Institute of Engineering and Applied Sciences.

Islamabad, Pakistan

August 30, 2024

Introduction:

The magnetic pendulum serves as a fascinating demonstration of non-linear dynamics, revealing complex behaviors such as chaos and bistable states. In this summer project, we focused on the experimental and theoretical analysis of this system. The project began with the repair and calibration of a malfunctioning magnetic pendulum setup, ensuring it could effectively demonstrate the intricate oscillations and chaotic motions characteristic of non-linear systems.

Through research into the individual components of the setup and rigorous experimentation, we explored the pendulum's sensitive dependence on initial conditions, which often leads to unpredictable, chaotic motion. We used Python programming for the theoretical modeling and graphical analysis, delving into the dynamics that drive the pendulum's movement. This analysis extended to solving non-linear equations that describe the system, further emphasizing the rich and complex behavior of magnetic pendulums.

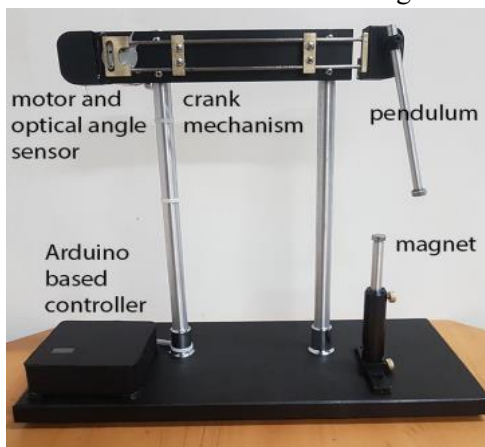
Project Objectives:

- To fix the faulty magnetic pendulum setup.
- To research the individual components of the setup.
- To run the setup for experimental and theoretical analysis.
- Analyzing the chaotic behavior of non-linear magnetic pendulum.
- Graphical analysis of magnetic pendulum using python.
- Using python to solve and model any non-linear problem.

Experimental work summary:

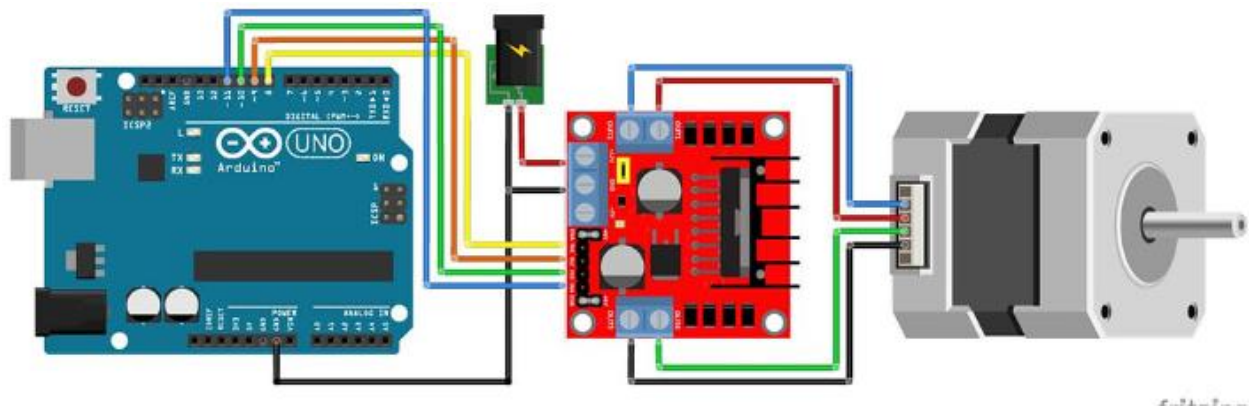
We started with examining the apparatus which was completely irresponsive. Our setup consisted of four basic components:

1. Arduino case
2. Stepper motor
3. LDR case
4. A pendulum rod hanging on a crank mechanism with disk magnets



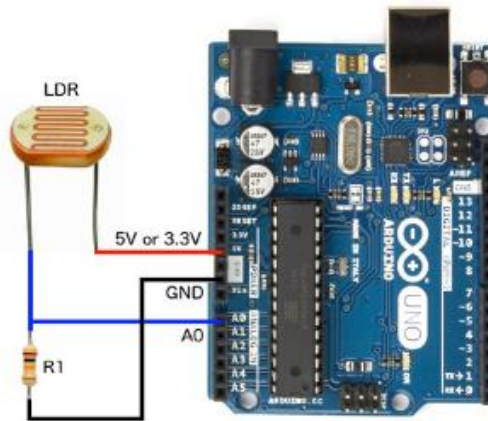
1. *Experimental Setup (vertical configuration)*

Our first objective was to run the pendulum through the motor, and then to take the output using LDR. We opened the Arduino case and found an Arduino mega and two drivers in it. *Arduino is basically a board able of reading input like light on a sensor, signal from a driver and turn it into output like activating a motor, turning on an LED.* We used Arduino to control the speed of stepper motor and to track the motion of pendulum using LDR. We understood the hardware and software of Arduino and wrote a code for operating the stepper motor. A pre-designed code for stepper motor is also available in Arduino software. We realized that there was no need of two drivers to control the devices and used only one driver to drive the motor. The Arduino Mega was also not working properly so we replaced it with an Arduino UNO. The difference between these two is just the quantity of devices they can control. Arduino Mega can control more devices. But for just two devices, Arduino UNO was enough which has 14 digital input/output pins and 6 analog input pins. We connected the motor to the driver and driver to the Arduino, the circuit diagram is given in the following figure demonstrating the connections:



2. Circuit diagram of stepper motor + Arduino + motor driver

We run the Arduino code but still the motor was not working so we suspected that the motor was malfunctioned. We uncovered the motor and looked for any fixable fault, but we were unable to fix the motor. Then we researched the specifications of the motor which were 0.9 degree per step and 6V supply. We replaced the motor with another stepper motor having similar specifications of 1.8 degree per step and nearly same voltage. Finally, we connected the working motor with the crack mechanism setup and the pendulum started swinging. Then comes the second part of our objective, the LDR which was responsive. LDR case contained a circuit in which LDR and LED light were in series while 2 resistors were connected to LDR in parallel. The LDR was connected to Arduino as shown in the circuit diagram in fig 3. Due to the motion of pendulum, the LED light intensity falling upon the LDR sensor was changing, from which we got the output data of LDR value. Numeric output and graph from the serial plotter could be seen at the Arduino software interface. At the end, the stepper motor and LDR's separate Arduino codes were combined into a single code, and we also calculated the velocity and momentum of pendulum in that final code.



3. Circuit diagram of LDR + Arduino UNO

Arduino final code:

```
#include <Stepper.h>

const int stepsPerRevolution = 200; // change this to fit the number of steps per revolution for your motor

const int ldrPin = A0;           // LDR connected to analog pin A0

// Initialize the stepper library on pins 8 through 11
Stepper myStepper(stepsPerRevolution, 8, 9, 10, 11);

// Variables to hold the previous readings and time
int previousLdrValue = 0;
unsigned long previousTime = 0;

void setup() {
    // Stepper motor setup
    myStepper.setSpeed(100); // Set the speed at 100 rpm

    // Initialize the serial port
    Serial.begin(9600);
}

void loop() {
    // 1. Stepper Motor Control

    // Step one revolution in one direction
```

```

//Serial.println("Clockwise rotation");

myStepper.step(stepsPerRevolution); // Move stepper motor one revolution clockwise

delay(5);

// 2. LDR Reading and Momentum Calculation

// Read the current LDR value

int currentLdrValue = analogRead(ldrPin);

// Get the current time

unsigned long currentTime = millis();

// Calculate the change in LDR value (this indicates a change in light intensity)

int deltaLdrValue = currentLdrValue - previousLdrValue;

// Calculate the change in time in seconds

float deltaTime = (currentTime - previousTime) / 1000.0; // Convert milliseconds to seconds

// If there is a change in light intensity, calculate velocity

if (deltaTime > 0) {

    // Estimate the velocity (change in light intensity per second)

    float velocity = deltaLdrValue / deltaTime;

    // Assuming a mass of the object (e.g., 16 g for simplicity)

    float mass = 0.16; // Adjust as needed for the actual object's mass

    float momentum = mass * velocity;

    // Print the results to the Serial Monitor

    Serial.print("LDR Value: ");

    Serial.println(currentLdrValue);

    Serial.print("Velocity: ");

    Serial.println(velocity);

    Serial.print("Momentum: ");

    Serial.println(momentum); }

```

```

// Update the previous readings and time

previousLdrValue = currentLdrValue;

previousTime = currentTime;

// Wait a short period before the next loop iteration

delay(0); // Adjust delay based on the required sampling rate

}

```

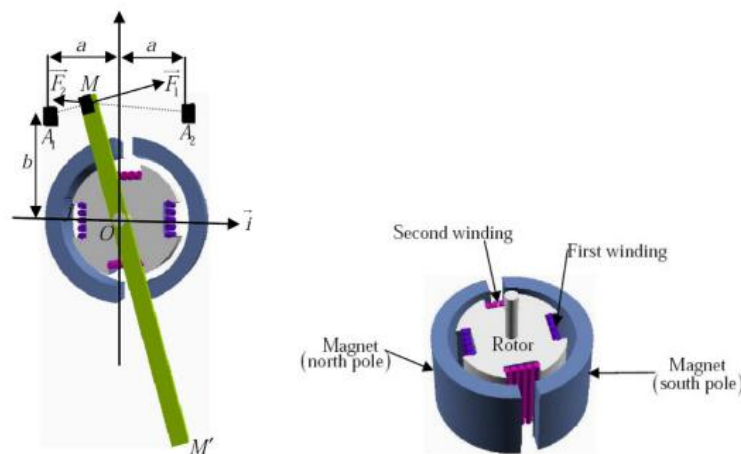
Experimental results:

After our efforts, the setup was finally working properly, the motion of pendulum was being processed by LDR. The speed of motor could be controlled using Arduino. LDR value, velocity and momentum of the pendulum was being recorded. We obtained graphs of 'LDR value' vs 't' and 'momentum' vs 'velocity'. Phase portraits and other graphs like bifurcation diagram and Lyapunov exponent could be observed using Phys logger but Phys logger code for our setup was missing so we couldn't simulate the experimental results in real time, but we did it theoretically.

Theoretical work summary:

We modelled the non-linear system of equations of magnetic pendulum on python. For equations, we consulted an article named as "Dynamics of a pendulum driven by a DC motor and magnetically controlled" by "B. Nanaa, S.B. Yamgouéa, R. Tchitnga b, P. Wofo". To do this rigorous task, we build our foundation by solving simple linear and non-linear ODEs using python, then system of ODEs, constructing directional field, solving special functions (Legendre and Bessel), generating different tables and plots, saving them in the hard disk of computer and then we moved towards our main objective.

Our theoretical model setup (shown in fig 4) was a bit different than experimental one. However, the principle on which setup was build was same.



4. Theoretical model setup (horizontal configuration)

Theoretical model was based on these six forces:

1. Repulsive magnetic force F_1
2. Repulsive magnetic force F_2
3. Sinusoidal driving force by motor
4. Damping force
5. Weight of magnet at point 'M'
6. Weight of pendulum

Governing system of equations:

$$\ddot{\theta} + \lambda_m \dot{\theta} + f(\theta) = \varepsilon_m \chi,$$

$$\dot{x} + \lambda_e x + \varepsilon_e \dot{\theta} \cos \theta = E \sin(\Omega \tau),$$

Parameters and functions required:

$$f(\theta) = \gamma_1 f_1(\theta) + \gamma_2 \sin(\theta) \quad \lambda_e = \frac{R}{L\omega_0} \quad E = \frac{E_m}{L\omega_0 I_0}$$

$$\Omega = \frac{\omega}{\omega_0} \quad \epsilon_e = \frac{hDB_m N}{LI_0} \quad \lambda_m = \frac{\beta_m}{J_m \omega_0}$$

$$\gamma_1 = \frac{\mu_0 q_1 q_2}{4\pi J_m \omega_0^2 l_1} \quad \epsilon_m = \frac{hDB_m N I_0}{J_m \omega_0^2} \quad \gamma_2 = \frac{g}{J_m \omega_0^2 (ml - ml_1 - m_0 l_1)}$$

These variable parameters depend upon some constant parameters. We faced problems in determining values of some parameters, so we mailed the author of article and he helped us in finding the values. The values found are mentioned below:

Parameters	Values	Parameters	Values
ℓ	$13.35 \cdot 10^{-2} \text{ m}$	ℓ_1	$5.80 \cdot 10^{-2} \text{ m}$
a	$2.02 \cdot 10^{-2} \text{ m}$	b	$5.44 \cdot 10^{-2} \text{ m}$
h	$9.85 \cdot 10^{-2} \text{ m}$	d	$0.15 \cdot 10^{-3} \text{ m}$
D	$8.12 \cdot 10^{-2} \text{ m}$	ℓ_g	$5.0 \cdot 10^{-3} \text{ m}$
m	$5.0 \cdot 10^{-3} \text{ Kg}$	β_m	$2.12 \cdot 10^{-4} \text{ Ns}$
q_1	5.19 Am	$m_{01} = m_0$	$5.50 \cdot 10^{-3} \text{ Kg}$
q_2	6.02 Am	m_{02}	$5.58 \cdot 10^{-3} \text{ Kg}$
φ	$1.72 \cdot 10^{-8} \text{ } \Omega\text{m}$	μ_r	985
N	20	B_m	0.73 T
R_1	$10K\Omega$	C_1	$100\mu\text{F}$
g	9.82 N/Kg	-	-

$$\lambda_m = 1.0099$$

$$f = 9.4427$$

$$\epsilon_m = 1.4471$$

$$\lambda_e = 3.2279$$

$$\epsilon_e = 20.7104$$

$$E = 291.6357$$

$$W = 5.6710$$

$$\Omega = 100 \text{ rad/s}$$

$$L = 0.792 \text{ m}$$

$$J_m = 7.6705e-5$$

$$\Delta_1 = 0.348$$

$$\Delta_2 = 0.937$$

$$\Gamma_1 = 7.024e-5$$

$$\Gamma_2 = 0.000748$$

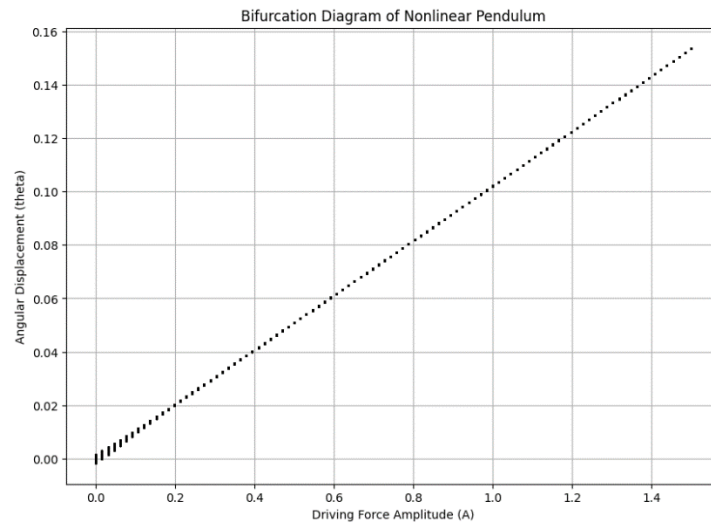
$$c_0 = 0.0069; c_1 = 0.096$$

We plotted graphs of “ $f(\theta)$ vs θ ”, “ X vs E_m ” and phase portraits i.e. “ θ vs $\dot{\theta}$ ”. Our graphs weren’t 100% accurate because of varying initial conditions and parameter values but they showed a nice approximation of chaotic behavior. Phase portraits obtained are shown in the theoretical results section.

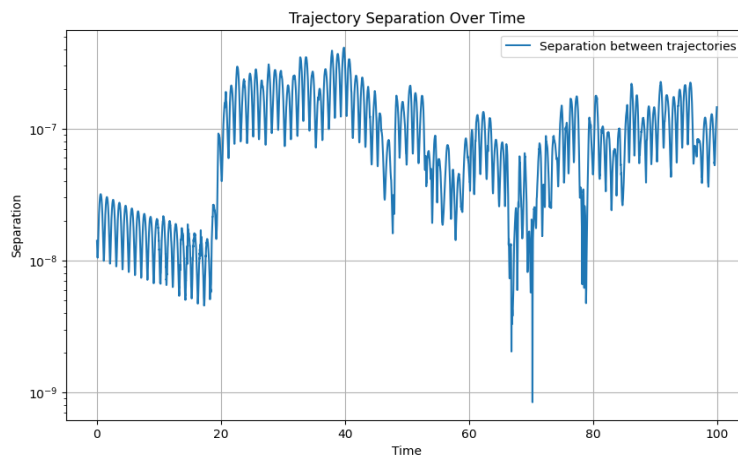
Moreover, at the end of our theoretical analysis, we made bifurcation diagram and Lyapunov exponent of a random forced damped non-linear pendulum using python, and the figures obtained were pretty accurate.

The governing equation of non-linear driven pendulum:

$$L\ddot{\theta} + b\dot{\theta} + g \sin \theta = F \cos \omega t$$



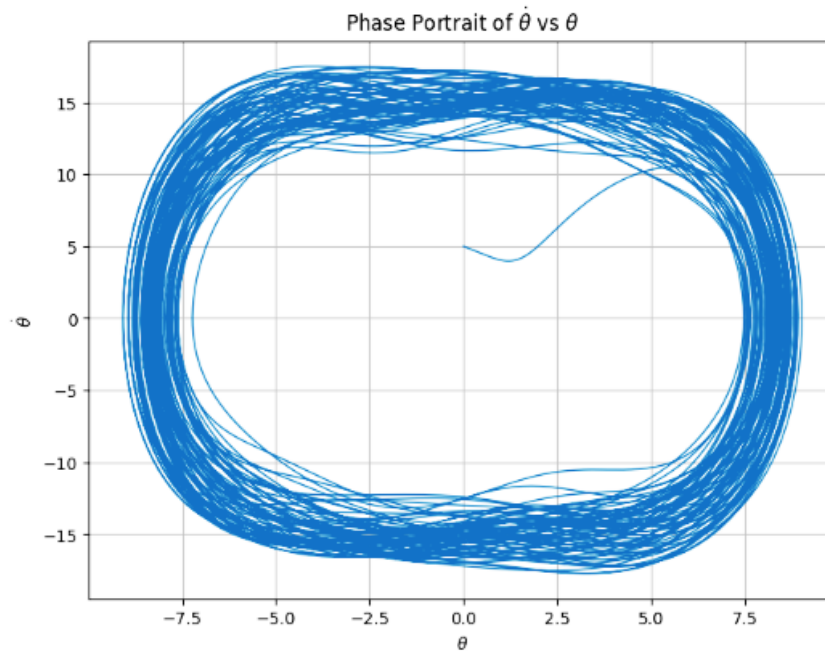
5. Bifurcation diagram of forced damped non-linear pendulum



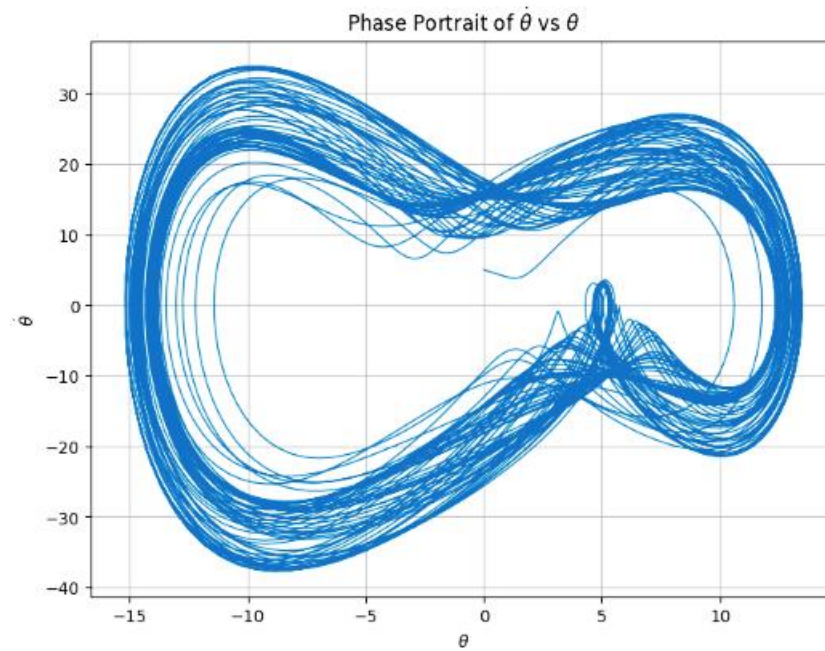
6. Lyapunov exponent plot of forced damped non-linear pendulum

Theoretical results:

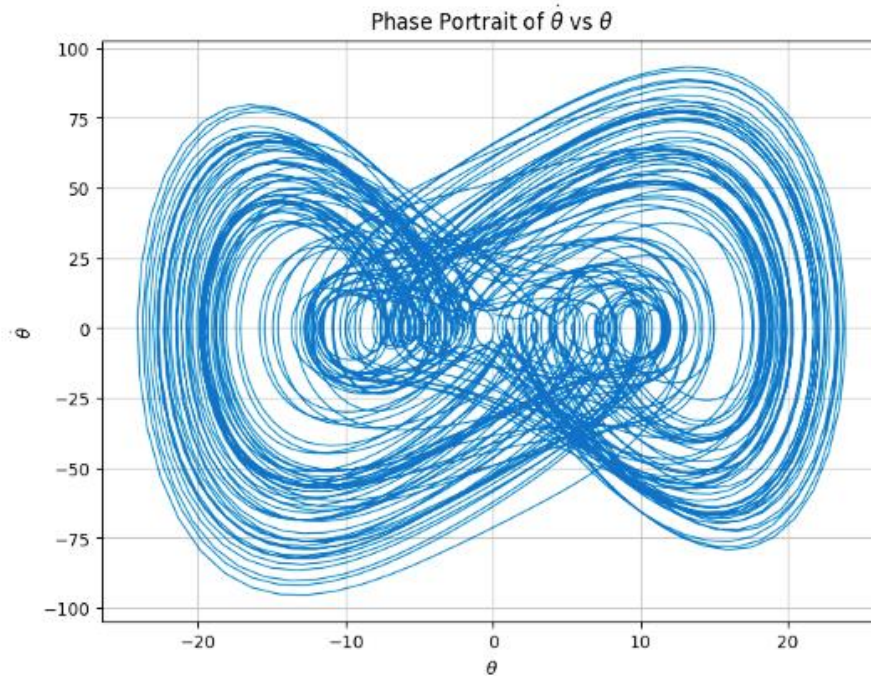
The phase portraits of magnetic pendulum obtained theoretically are shown below:



7. Phase portrait at high frequency



8. Phase portrait at moderate frequency



9. Phase portrait at low frequency

Conclusion:

The completion of this summer project on the experimental and theoretical analysis of a magnetic pendulum provided a comprehensive understanding of non-linear dynamics, particularly focusing on chaotic behavior and bistable states. Through extensive experimental work, we successfully repaired the pendulum setup, which had initially been faulty. By leveraging Arduino technology and implementing a precise stepper motor system, we managed to capture the pendulum's oscillations and accurately calculate physical properties such as velocity and momentum. Despite some challenges in the experimental apparatus and inability to visualize the experimental output, we gained significant insights by tracking and controlling the pendulum's motion using light-dependent resistor (LDR).

In parallel, our theoretical analysis allowed us to delve deeper into the governing equations of the magnetic pendulum. Through Python modeling, we simulated non-linear behaviors, including the production of phase portraits and bifurcation diagrams, which aligned well with established theoretical predictions. The exploration of complex phenomena, such as chaotic oscillations, underscored the richness of this non-linear system, even if some experimental data couldn't be fully replicated due to lack of resources. Ultimately, this project has demonstrated the intricate interplay between theory and experiment in understanding chaotic systems, providing a solid foundation for future studies in non-linear dynamics.